XSPARKS

Engineering Excellence Division

FULL STACK ENGINEER TRAINING PROGRAM

Comprehensive 4-Week Accelerated Learning Curriculum

Version 3.0

Effective Date: 06 January 2026

Document ID	GTC-ENG-TRN-2025-001
Classification	Internal Use Only
Document Owner	Chief Technology Officer
Approved By	Global Training & Development Board
Review Cycle	Quarterly
Next Review	April 2026

 |

TABLE OF CONTENTS

DOCUMENT CONTROL

Revision History

Version	Date	Author	Changes
1.0	Oct 2025	Engineering Team	Initial document creation
2.0	Dec 2025	Training Board	Added AI/ML modules, Docker optimization
3.0	Jan 2026	Engineering Team	Added global delivery framework, governance, accessibility standards

EXECUTIVE SUMMARY

This document establishes the Global Full Stack Engineer Training Program, a comprehensive 4-week accelerated curriculum designed to develop production-ready engineers capable of contributing to enterprise AI-powered projects across all global business units.

The program addresses the critical need for skilled full stack engineers who can navigate modern development ecosystems, including frontend frameworks, backend architectures, cloud infrastructure, DevOps practices, and cutting-edge AI/ML integration. Upon completion, participants will possess the technical competencies and practical experience required to deliver value from day one.

Program Overview

Program Duration	4 Weeks (20 Working Days)
Daily Commitment	8 Hours (Including Labs)
Delivery Format	Hybrid (In-Person + Virtual)
Maximum Cohort Size	15 Participants per Region
Target Audience	New Full Stack Engineers
Certification	GTC Full Stack Engineer Certified

Strategic Alignment

This training program directly supports the following corporate strategic objectives:

- Digital Transformation Initiative: Accelerating AI-powered product development

 |

- Talent Development Strategy: Building internal engineering capabilities
- Operational Excellence: Standardizing engineering practices globally
- Innovation Roadmap: Enabling multi-agent systems and LLM integration

GOVERNANCE FRAMEWORK

Program Ownership

Role	Responsibility	Escalation Authority
Program Sponsor (CTO)	Strategic direction, budget approval, executive oversight	Board Level
Program Director	Curriculum design, quality assurance, vendor management	CTO
Regional Training Leads	Local delivery, mentor coordination, facility management	Program Director
Technical Mentors	Daily instruction, hands-on guidance, assessment	Regional Lead
HR Business Partners	Participant selection, onboarding, career path integration	Regional Lead

Steering Committee

The Training Program Steering Committee convenes monthly to review program effectiveness, address challenges, and approve curriculum updates.

Committee Member	Department	Meeting Cadence
Chief Technology Officer	Technology	Monthly
VP Engineering	Engineering	Monthly
Head of Global Training	HR/L&D	Monthly
Regional Engineering Directors	Engineering (All Regions)	Monthly
Finance Representative	Finance	Quarterly

Escalation Matrix

Issue Type	First Contact	Escalation Path	Resolution SLA
Technical Content	Technical Mentor	Regional Lead > Program Director	24 hours
Participant Performance	Technical Mentor	Regional Lead > HR BP	48 hours

 |

Issue Type	First Contact	Escalation Path	Resolution SLA
Facility/Logistics	Regional Admin	Regional Lead > Operations	4 hours
System Access	IT Help Desk	Regional IT > Global IT	2 hours
Curriculum Gaps	Regional Lead	Program Director > Steering Committee	1 week

PREREQUISITES & ASSESSMENT

Entry Requirements

All participants must meet the following prerequisites before program enrollment:

Requirement	Description	Verification Method
Educational Background	Bachelor's degree in CS, Engineering, or equivalent experience	HR Verification
Programming Fundamentals	Proficiency in at least one programming language (Python, JavaScript, or Java)	Technical Assessment
Web Development Basics	Understanding of HTML, CSS, HTTP protocols	Technical Assessment
Database Concepts	Basic SQL knowledge, understanding of relational databases	Technical Assessment
Version Control	Familiarity with Git basic operations	Practical Exercise
English Proficiency	B2 level or above (for non-native regions)	Language Assessment

Pre-Program Assessment

A standardized assessment is administered 2 weeks prior to program start to ensure participant readiness and identify areas requiring additional support.

Assessment Component	Duration	Passing Score	Retake Policy
Programming Logic Test	60 minutes	70%	1 retake allowed
Web Fundamentals Quiz	30 minutes	75%	1 retake allowed
SQL Proficiency Test	45 minutes	65%	1 retake allowed
Practical Coding Exercise	90 minutes	Pass/Fail	Review with mentor

Participants who do not meet minimum scores will be provided with pre-work materials and assigned a study partner for the 2-week preparation period.

 |

GLOBAL DELIVERY FRAMEWORK

Regional Adaptations

While maintaining curriculum consistency, the following regional adaptations ensure effective delivery across all global locations:

Region	Primary Location	Time Zone	Language Support	Local Considerations
Americas	Pittsburgh	CST / BRT	English	US cloud region focus
APAC	Delhi / Chandigarh / Mumbai / Bangalore	SGT / IST	English, Hindi	Regional cloud providers

Delivery Modes

Mode	Description	When Used	Technology
In-Person Labs	Hands-on sessions with mentor supervision	Complex technical modules, assessments	Physical lab environment
Virtual Instructor-Led	Live sessions with screen sharing	Theory, demonstrations, Q&A	MS Teams / Zoom
Self-Paced Learning	Pre-recorded content and exercises	Foundational concepts, review	LMS Platform
Hybrid	Combined in-person and virtual participants	Cross-regional collaboration	Teams Rooms + Virtual

Cross-Regional Collaboration Sessions

Weekly global sessions connect participants across regions for knowledge sharing and collaborative exercises:

Session	Day/Time (UTC)	Duration	Purpose
Global Standup	Everyday 10:00 IST	30 min	Progress sharing, Q&A
Tech Talk Thursday	Thursday 15:00 IST	60 min	Expert presentations, demos
Capstone Review	Friday 16:00 UTC (Week 4)	90 min	Project presentations

 |

LEARNING OBJECTIVES

Upon successful completion of this program, participants will demonstrate proficiency in the following competency areas:

Technical Competencies

Domain	Skills Acquired	Proficiency Level
Frontend Development	React, state management, performance optimization, testing	Intermediate
Backend Development	Node.js, Python FastAPI, RESTful APIs, GraphQL, WebSocket	Intermediate
Database Systems	PostgreSQL, MongoDB, vector databases, caching strategies	Intermediate
Cloud Infrastructure	AWS, GCP, serverless, container orchestration	Foundational
DevOps & CI/CD	Docker, Kubernetes, Jenkins, GitHub Actions	Foundational
AI/ML Integration	LLMs, RAG, multi-agent systems, prompt engineering	Foundational
Security	Authentication, authorization, secure coding practices	Foundational

Professional Competencies

Competency	Description	Assessment Method
Technical Communication	Document code, write technical specs, present solutions	Code reviews, presentations
Collaboration	Work effectively in cross-functional teams	Peer feedback, team exercises
Problem Solving	Analyze requirements, design solutions, debug issues	Capstone project, assessments
Continuous Learning	Self-direct learning, stay current with technology	Learning journal, participation

 |

WEEK 1: FOUNDATIONS & CORE DEVELOPMENT

Theme: Building Blocks of Full Stack Development

Day 1: Frontend Optimization & Code Quality

Learning Objectives:

- Understand React Context API and state management patterns
- Implement lazy loading and code splitting for performance
- Master pagination techniques (cursor-based and offset-based)
- Configure ESLint and Prettier for code quality enforcement

Daily Schedule:

Time	Activity	Duration	Delivery
09:00-10:00	React Context API and useContext hook	1 hr	Lecture + Demo
10:00-11:30	Lazy loading with React.lazy() and Suspense	1.5 hr	Hands-on Lab
11:30-12:30	Lunch Break	1 hr	-
12:30-14:30	Building paginated data table with infinite scroll	2 hr	Hands-on Lab
14:30-15:30	ESLint and Prettier configuration	1 hr	Hands-on Lab
15:30-17:00	Performance audit with Chrome DevTools and Lighthouse	1.5 hr	Lab + Review

Daily Completion Checklist:

- ☐ Context API implementation completed with working example
- ☐ Lazy loading configured in at least 3 components
- ☐ Pagination system working with API integration
- ☐ ESLint configuration file created and validated
- ☐ Performance score >90 on Lighthouse audit
- ☐ Code committed to version control with proper commit messages

Day 2: Backend Fundamentals - Node.js

Learning Objectives:

- Set up Express.js server with middleware architecture
- Implement RESTful API design principles
- Apply JWT-based authentication and session management
- Implement request validation and error handling

Daily Schedule:

 |

Time	Activity	Duration	Delivery
09:00-10:00	Initialize Node.js project with Express.js	1 hr	Hands-on Lab
10:00-12:00	Create RESTful endpoints for CRUD operations	2 hr	Hands-on Lab
12:00-13:00	Lunch Break	1 hr	-
13:00-15:00	JWT authentication middleware implementation	2 hr	Lecture + Lab
15:00-16:30	Request validation and error handling	1.5 hr	Hands-on Lab
16:30-17:30	API documentation with Swagger/OpenAPI	1 hr	Lab + Review

Daily Completion Checklist:

- ☐ Express server running with structured folder architecture
- ☐ Minimum 5 RESTful endpoints created (GET, POST, PUT, DELETE)
- ☐ Environment variables properly configured (.env file)
- ☐ JWT authentication working with token generation and verification
- ☐ Error handling middleware implemented
- ☐ API documentation generated and accessible

 |

Day 3: Backend Fundamentals - Python FastAPI

Learning Objectives:

- Understand FastAPI framework and async programming patterns
- Implement Pydantic models for data validation
- Apply dependency injection patterns and OAuth2 authentication
- Compare FastAPI vs Node.js performance characteristics

Daily Schedule:

Time	Activity	Duration	Delivery
09:00-10:00	Set up FastAPI project structure	1 hr	Hands-on Lab
10:00-12:00	Create CRUD endpoints with async functions	2 hr	Hands-on Lab
12:00-13:00	Lunch Break	1 hr	-
13:00-14:30	Pydantic models for request/response validation	1.5 hr	Lecture + Lab
14:30-16:30	OAuth2 password flow authentication	2 hr	Hands-on Lab
16:30-17:30	Performance comparison: FastAPI vs Express	1 hr	Lab + Discussion

Daily Completion Checklist:

- ☐ FastAPI application running with auto-generated documentation
- ☐ Async endpoints implemented for all CRUD operations
- ☐ Pydantic validation working on all endpoints
- ☐ OAuth2 authentication configured and tested
- ☐ CORS properly configured
- ☐ Performance benchmarks documented

Day 4: Version Control & Testing Foundations

Learning Objectives:

- Master Git workflows (branching, merging, rebasing)
- Implement browser automation testing with Playwright
- Set up end-to-end testing with Puppeteer
- Create test automation strategies for CI integration

Daily Schedule:

Time	Activity	Duration	Delivery
09:00-10:30	Git fundamentals review and advanced commands	1.5 hr	Lecture + Exercises
10:30-11:30	Branching strategies (Git Flow, trunk-based)	1 hr	Workshop
11:30-12:30	Lunch Break	1 hr	-

 |

Time	Activity	Duration	Delivery
12:30-14:30	Set up Playwright test suite for frontend	2 hr	Hands-on Lab
14:30-16:30	Implement Puppeteer tests for critical user flows	2 hr	Hands-on Lab
16:30-17:30	Configure test reporting and CI integration	1 hr	Lab + Review

Daily Completion Checklist:

- ☐ Git workflow document created with team standards
- ☐ Branching strategy demonstrated with feature branches
- ☐ Playwright tests written for minimum 5 user scenarios
- ☐ Puppeteer tests automated for login and data operations
- ☐ Test reports generated with screenshots
- ☐ All tests passing in local environment

 |

Day 5: System Design & Database Design Fundamentals

Learning Objectives:

- Understand system design principles (scalability, reliability, maintainability)
- Design database schemas for relational and NoSQL databases
- Apply normalization/denormalization strategies and indexing
- Understand CAP theorem and database trade-offs

Daily Schedule:

Time	Activity	Duration	Delivery
09:00-11:00	System design fundamentals (load balancing, caching, sharding)	2 hr	Lecture
11:00-12:30	Design schema for e-commerce application	1.5 hr	Workshop
12:30-13:30	Lunch Break	1 hr	-
13:30-14:30	Database normalization exercises	1 hr	Hands-on Lab
14:30-16:00	PostgreSQL indexing strategies	1.5 hr	Hands-on Lab
16:00-17:30	MongoDB vs PostgreSQL comparison + Architecture diagram	1.5 hr	Lab + Presentation

Daily Completion Checklist:

- ☐ System design document created with architecture diagram
- ☐ Database schema designed (ERD for PostgreSQL)
- ☐ Normalization level identified and justified (3NF/BCNF)
- ☐ Indexes identified for performance optimization
- ☐ NoSQL schema design completed for MongoDB
- ☐ Architecture diagram includes caching and load balancing layers

 |

WEEK 2: APIs, DATABASES & AUTHENTICATION

Theme: Data Management & Integration

Day 6: API Protocols - REST, GraphQL, WebSocket

Learning Objectives:

- Master RESTful API design patterns and best practices
- Implement GraphQL server with queries and mutations
- Set up WebSocket for real-time bidirectional communication
- Compare API protocol trade-offs for different use cases

Daily Completion Checklist:

- ☐ REST API with proper HTTP status codes and versioning
- ☐ GraphQL server with minimum 5 queries and 3 mutations
- ☐ WebSocket server handling real-time messages
- ☐ Protocol comparison document created with recommendations

Day 7: Database Implementation - SQL (PostgreSQL)

Learning Objectives:

- Set up PostgreSQL database with connection pooling
- Write complex queries with joins, subqueries, and CTEs
- Implement database migrations with version control
- Apply query optimization using EXPLAIN ANALYZE

Daily Completion Checklist:

- ☐ PostgreSQL database configured with proper user permissions
- ☐ Schema migrations working (up/down migrations)
- ☐ Complex queries written (minimum 5 with JOINs)
- ☐ Query performance benchmarks documented

Day 8: Database Implementation - NoSQL & Vector Databases

Learning Objectives:

- Understand MongoDB document model and operations
- Implement vector databases (Weaviate, ChromaDB) for AI applications
- Work with cloud storage (AWS S3, GCP Cloud Storage)
- Implement semantic search with vector embeddings

Daily Completion Checklist:

 |

- ☐ MongoDB collections created with proper validation rules
- ☐ Vector database configured and tested with sample data
- ☐ Embedding generation pipeline implemented
- ☐ Semantic search working with similarity threshold
- ☐ S3/GCS integration complete with signed URLs

 |

Day 9: Caching Mechanisms & Multi-threading

Learning Objectives:

- Implement L1 (in-memory), L2 (Redis), L3 (CDN) caching strategies
- Understand cache invalidation strategies and patterns
- Apply Node.js clustering and worker threads for CPU-intensive tasks
- Implement Python multiprocessing and asyncio patterns

Daily Completion Checklist:

- ☐ L1 cache implemented with TTL and eviction policy
- ☐ Redis configured with cache-aside pattern
- ☐ CDN caching rules configured (CloudFlare)
- ☐ Worker threads processing background jobs
- ☐ Cache performance metrics documented (hit rate >80%)

Day 10: Authentication & Network Protocols

Learning Objectives:

- Implement Auth0 and Firebase Authentication integration
- Configure SMTP email sending and IMAP email reading
- Understand SSL/TLS, SSH, HTTPS, and CORS protocols
- Apply security best practices for web applications

Daily Completion Checklist:

- ☐ Auth0 integration working with role-based access control
- ☐ Firebase Auth configured (Email, Google, GitHub providers)
- ☐ Email sending functional with HTML templates
- ☐ SSL certificate installed and HTTPS working
- ☐ CORS configured with whitelisted origins

WEEK 3: DEPLOYMENT, DEVOPS & DOCKER OPTIMIZATION

Theme: Infrastructure & Automation

Day 11: Cloud Deployment - AWS

Learning Objectives:

- Deploy applications on AWS EC2 with proper configuration
- Configure AWS RDS for managed database services
- Set up AWS S3 with CloudFront CDN for static assets
- Implement AWS Lambda serverless functions with API Gateway
- Configure VPC, security groups, and IAM roles

Daily Completion Checklist:

- ☐ EC2 instance running with application deployed
- ☐ RDS database connected and migrations run
- ☐ S3 bucket serving static files through CloudFront
- ☐ Lambda function deployed and triggered via API Gateway
- ☐ Security groups configured (least privilege principle)
- ☐ Cost estimation documented

Day 12: Cloud Deployment - GCP & Serverless Platforms

Learning Objectives:

- Deploy on Google Cloud Platform (Compute Engine, Cloud SQL)
- Use serverless platforms (Vercel, Render, Netlify)
- Deploy on Digital Ocean for cost-effective hosting
- Compare cloud providers for different use cases

Daily Completion Checklist:

- ☐ GCP deployment complete and tested
- ☐ Digital Ocean deployment operational
- ☐ Vercel deployment with preview URLs
- ☐ Cost and performance comparison completed

Day 13: Docker & Container Optimization

Learning Objectives:

- Write production-ready Dockerfiles with multi-stage builds
- Optimize Docker images for size and performance
- Implement Docker layer caching strategies

 |

- Apply security best practices for containers

Docker Optimization Techniques:

- Use multi-stage builds to separate build and runtime dependencies
- Choose minimal base images (Alpine, distroless, scratch)
- Order Dockerfile instructions to maximize layer caching
- Use .dockerignore to exclude unnecessary files
- Combine RUN commands to reduce layers

Daily Completion Checklist:

- ☐ Dockerfiles created with multi-stage builds
- ☐ Image size optimized (document before/after sizes)
- ☐ Security scan completed (zero critical vulnerabilities)
- ☐ Images pushed to container registry

Day 14: Kubernetes & Container Orchestration

Learning Objectives:

- Understand Kubernetes architecture and components
- Create Deployment, Service, and Ingress manifests
- Implement ConfigMaps, Secrets, and environment management
- Set up Horizontal Pod Autoscaling for dynamic scaling

Daily Completion Checklist:

- ☐ Kubernetes cluster operational (Minikube/Kind)
- ☐ Deployment manifests applied successfully
- ☐ Services exposing applications correctly
- ☐ HPA scaling based on CPU/memory metrics
- ☐ Rolling update performed successfully

Day 15: CI/CD Pipelines

Learning Objectives:

- Set up Jenkins pipeline with Jenkinsfile
- Configure GitHub Actions workflows for automation
- Implement automated testing in CI pipeline
- Set up deployment automation with rollback strategies

Daily Completion Checklist:

- ☐ Jenkins pipeline running successfully
- ☐ GitHub Actions workflow triggered on push/PR
- ☐ All tests running in CI environment
- ☐ Automated deployment to staging environment
- ☐ Notifications configured for build status
- ☐ Deployment rollback strategy documented

WEEK 4: ADVANCED AI/ML & INFRASTRUCTURE

Theme: AI Integration, Multi-Agent Systems & LLM Engineering

Day 16: Infrastructure as Code & Observability

Learning Objectives:

- Write AWS CDK code and Terraform configurations
- Set up Prometheus for metrics collection
- Configure Grafana dashboards for visualization
- Implement Loki for centralized log aggregation

Daily Completion Checklist:

- ☐ AWS CDK/Terraform deployed successfully
- ☐ Prometheus collecting metrics from all services
- ☐ Grafana dashboards showing key metrics
- ☐ Alert rules configured with runbooks

Day 17: LLM Fundamentals & Model Types

Learning Objectives:

- Understand different LLM model architectures and types
- Differentiate between thinking and non-thinking models
- Explore model quantization and optimization techniques
- Set up local LLM inference with Ollama

Model Type Comparison:

Model Type	Characteristics	Use Cases	Examples
Thinking Models	Chain-of-thought reasoning, higher latency	Complex problem-solving	o1, o3-mini, Claude Extended
Non-Thinking Models	Direct response, faster inference	General chat, quick tasks	GPT-4, Claude Sonnet, Gemini
Coding Models	Trained on code repositories	Code completion, generation	CodeLlama, StarCoder
Task-Based Models	Optimized for specific tasks	Classification, summarization	BERT variants, T5

Daily Completion Checklist:

- ☐ Ollama installed with 4+ different model types
- ☐ Model performance comparison completed
- ☐ Quantization impact documented
- ☐ Model selection criteria documented

 |

Day 18: Prompt Engineering & Context Management

Learning Objectives:

- Master prompt engineering techniques and best practices
- Understand context window management and optimization
- Build RAG (Retrieval-Augmented Generation) systems
- Use DSPy for programmatic prompt optimization

Prompt Engineering Techniques:

- Clear Instructions: Be specific and explicit about requirements
- Few-Shot Learning: Provide examples in prompt for guidance
- Chain-of-Thought: Ask model to explain reasoning steps
- Role Assignment: Define persona ("You are an expert...")
- Output Formatting: Specify JSON, markdown, or other formats

Daily Completion Checklist:

- ☐ Prompt templates created and tested
- ☐ RAG pipeline implemented end-to-end
- ☐ DSPy optimization examples working
- ☐ Few-shot examples improve accuracy by 20%+

Day 19: Multi-Agent Systems & Agent Memory

Learning Objectives:

- Understand multi-agent orchestration patterns
- Implement Agent-to-Agent (A2A) communication protocol
- Build agent memory systems (hot/cold, short/long-term)
- Use LangGraph, LangChain, LangSmith, and Langfuse

Agent Memory Architecture:

Memory Type	Storage	Access Speed	Persistence	Use Case
Hot Memory	In-memory	Instant	Session-based	Current conversation context
Cold Memory	Vector DB	Fast (semantic)	Persistent	Historical conversations
Working Memory	Cache (Redis)	Very fast	Temporary	Active task context

Daily Completion Checklist:

- ☐ Hot memory system storing current session context
- ☐ Cold memory retrieving relevant historical data
- ☐ LangGraph workflow orchestrating 3+ agents
- ☐ LangSmith/Langfuse monitoring operational

 |

Day 20: Multimodal AI & MCP Integration

Learning Objectives:

- Implement multimodal AI (text, image, audio, video)
- Build speech-to-text (STT) and text-to-speech (TTS) pipelines
- Understand and implement Model Context Protocol (MCP)
- Implement queue management (RabbitMQ, SQS)

Multimodal Processing Overview:

Modality	Processing Type	Technologies	Output
Text	LLM inference, embeddings, NER	GPT-4, Claude, BERT	Generation, classification
Image	Generation, understanding, detection	CLIP, Stable Diffusion, YOLO	Images, labels, bounding boxes
Audio	Speech-to-text, text-to-speech	Whisper, ElevenLabs	Transcripts, audio files
Video	Frame extraction, scene detection	FFmpeg, custom models	Summaries, clips

Daily Completion Checklist:

- ☐ Multimodal pipeline processing all 4 modalities
- ☐ STT and TTS working in production
- ☐ MCP server exposing custom tools
- ☐ MCP client connected to AI models
- ☐ RabbitMQ/SQS processing async tasks

CAPSTONE PROJECT

No-Code AI Model Fine-Tuning Platform

Project Overview

The capstone project challenges participants to build a comprehensive no-code platform for AI model fine-tuning. This project integrates all skills acquired throughout the program, demonstrating proficiency in full stack development, AI/ML integration, and DevOps practices.

The platform enables users without coding expertise to fine-tune large language models using LoRA/QLoRA techniques through an intuitive 7-step workflow interface.

Technical Requirements

Component	Technology Stack	Deliverable
Frontend	React, TypeScript, TailwindCSS	Complete UI with 7 workflow steps
Backend	FastAPI, Python 3.11+	REST API with WebSocket support
Database	MongoDB + Vector DB (Weaviate/ChromaDB)	Schema design, migrations
AI Integration	Hugging Face, Kaggle APIs	Dataset import, model registry
Fine-Tuning	LoRA/QLoRA, PEFT library	Training pipeline with checkpoints
Queue System	RabbitMQ or AWS SQS	Async job processing
Monitoring	Prometheus, Grafana	Real-time metrics dashboard
Deployment	Docker, Kubernetes	Production-ready deployment

Platform Workflow Steps

Step	Function	Key Features
1. Dataset Selection	Import training data	Hugging Face/Kaggle integration, upload support
2. Data Preprocessing	Clean and prepare data	Tokenization, formatting, validation
3. Base Model Selection	Choose foundation model	Model browser, compatibility check
4. Training Configuration	Set hyperparameters	LoRA rank, learning rate, epochs
5. Training Execution	Run fine-tuning job	Progress tracking, checkpoint saves
6. Evaluation	Assess model performance	Metrics visualization, comparison
7. Export & Deploy	Package trained model	Download, API deployment options

 |

Capstone Deliverables Checklist

- ☐ Complete frontend with all 7 workflow steps functional
- ☐ FastAPI backend with REST endpoints and WebSocket
- ☐ Database schema implemented (MongoDB + Vector DB)
- ☐ Hugging Face and Kaggle API integration working
- ☐ LoRA/QLoRA fine-tuning pipeline operational
- ☐ Real-time WebSocket progress updates
- ☐ Queue system processing training jobs
- ☐ Training metrics visualization dashboard
- ☐ Model evaluation and comparison features
- ☐ Export and deployment functionality
- ☐ Docker containerization with optimization (<200MB)
- ☐ Kubernetes deployment manifests
- ☐ CI/CD pipeline for automated deployment
- ☐ Prometheus/Grafana monitoring configured
- ☐ Complete documentation and user guide
- ☐ Demo video (5-10 minutes)
- ☐ Deployed and accessible URL provided

ASSESSMENT & CERTIFICATION

Assessment Criteria

Category	Weight	Components	Minimum Score
Technical Proficiency	30%	Code quality, system design, problem-solving	70%
Capstone Project	40%	Feature completeness, architecture, documentation	75%
DevOps & AI/ML Skills	20%	Deployments, CI/CD, AI integration quality	70%
Best Practices	10%	Security, testing coverage, code reviews	80%

Certification Requirements

To receive the XSPARKS Full Stack Engineer Certification, participants must complete all of the following:

- ☐ Minimum 90% daily report checklist completion
- ☐ All hands-on projects submitted and reviewed
- ☐ Capstone project deployed and operational
- ☐ Final presentation delivered to review panel
- ☐ Minimum 10 code reviews participated in
- ☐ All Docker images optimized (<200MB for Node.js apps)
- ☐ AI pipelines functional with proper error handling
- ☐ Minimum 80% test coverage on all projects

Certification Levels

Level	Overall Score	Benefits	Badge Color
Certified	70-79%	Standard certification, digital badge	Bronze
Certified with Distinction	80-89%	Priority project assignments, mentorship opportunity	Silver
Certified with Honors	90-100%	Leadership track eligibility, conference sponsorship	Gold

 |

Mentor Responsibilities

Activity	Frequency	Duration	Purpose
Daily Standup	Daily	15 min	Progress check, blockers, priorities
Technical Guidance	As needed	Varies	Hands-on problem solving
Code Review	Daily	30 min	Quality feedback, best practices
Assessment Review	Weekly	45 min	Performance evaluation, feedback

SUPPORT STRUCTURE

Help Resources

Resource	Access Method	Availability	Response SLA
Technical Help Desk	whatsapp #training-support	24/7	4 hours
Mentor Hotline	Direct Slack DM/Whatsapp	Program hours	1 hour

Frequently Asked Questions

Q: What if I fall behind on the daily schedule?

A: Contact your mentor immediately. Additional support sessions can be arranged, and the schedule can be adjusted for valid circumstances. Consistent communication is key.

Q: Can I access training materials after program completion?

A: Yes.

Q: What happens if I don’t pass the assessment?

A: Participants who don’t meet minimum requirements can retake specific modules within 30 days. A remediation plan will be created with your mentor and HR.

 |

STANDARDS & BEST PRACTICES

Code Quality Standards

- Minimum 80% test coverage on all submitted code
- All pull requests require peer code review before merge
- ESLint/Pylint must pass without errors
- No critical security vulnerabilities in dependencies
- API documentation required for all public endpoints
- Type hints required for all Python code
- TypeScript required for all frontend code

Security Guidelines

- Never commit secrets, API keys, or credentials to version control
- Use environment variables for all configuration
- Implement input validation on all endpoints
- Apply principle of least privilege for all access controls
- Regular dependency updates (weekly during program)
- Rate limiting required on all public API endpoints

AI/ML Best Practices

- Version control for all datasets and models (DVC or similar)
- Experiment tracking with MLflow or Weights & Biases
- Model cards required for documentation
- Bias and fairness testing on all deployed models
- Cost monitoring for all AI API usage
- Prompt injection prevention measures required
- Content moderation filters on all user-facing AI outputs

Docker Optimization Checklist

- ☐ Multi-stage builds implemented
- ☐ Alpine or distroless base images used
- ☐ .dockerignore configured
- ☐ Build cache optimized
- ☐ Security scanning passed (Trivy)
- ☐ Image size documented (before/after)
- ☐ Health checks configured

 |

DAILY REPORTING TEMPLATE

All participants must submit a daily report by 5:00 PM local time. Reports are submitted via mail and reviewed by mentors within 24 hours.

DAILY PROGRESS REPORT

Date: [DD/MM/YYYY]
Day: [Day Number] - [Topic]
Participant Name: _____
Mentor: _____

LEARNING SUMMARY
• Key concept 1: _____
• Key concept 2: _____
• Key concept 3: _____

CHALLENGES FACED
• Challenge: _____ Resolution: _____

COMPLETED TASKS
☐ Task 1: _____
☐ Task 2: _____
☐ Task 3: _____

CODE REPOSITORY LINK: _____

QUESTIONS FOR REVIEW
1. _____
2. _____

TOMORROW'S PREPARATION: _____

Mentor Sign-off: _____ Date: _____

 |

RECOMMENDED RESOURCES

Official Documentation

- Hugging Face Transformers Documentation
- PEFT (Parameter-Efficient Fine-Tuning) Library
- LangChain and LangGraph Documentation
- DSPy Documentation
- Ollama Documentation
- Node.js, Express.js, FastAPI Official Docs
- Docker Best Practices Guide
- Kubernetes Official Documentation

Online Platforms

- Hugging Face Hub - Model and dataset repository
- Kaggle - Datasets and competitions
- Papers with Code - Research implementations
- GitHub - Version control and collaboration
- Stack Overflow - Community support

LLM Model Selection Guide

Task Type	Recommended Models	Notes
Code Generation	CodeLlama 7B/13B/34B, StarCoder2, DeepSeek Coder	Best for IDE integration
General Chat	LLaMA 3 8B/70B, Mistral 7B, Gemma 7B	Good balance of quality/speed
Thinking/Reasoning	o1-mini, Claude 3.5 Sonnet, Mixtral 8x7B	Complex problem solving
Classification	BERT variants, DistilBERT	Fast, specialized
Summarization	T5, BART, Pegasus	Document processing
Transcription	Whisper (various sizes)	Audio to text

 |

APPENDIX

A. Glossary of Terms

Term	Definition
A2A	Agent-to-Agent protocol for multi-agent communication
CDN	Content Delivery Network for distributed static asset delivery
CI/CD	Continuous Integration / Continuous Deployment
CTE	Common Table Expression in SQL
GDPR	General Data Protection Regulation (EU)
HPA	Horizontal Pod Autoscaler in Kubernetes
IaC	Infrastructure as Code
LLM	Large Language Model
LoRA	Low-Rank Adaptation for efficient model fine-tuning
MCP	Model Context Protocol
PEFT	Parameter-Efficient Fine-Tuning
QLoRA	Quantized LoRA for memory-efficient fine-tuning
RAG	Retrieval-Augmented Generation
SLA	Service Level Agreement
STT/TTS	Speech-to-Text / Text-to-Speech

B. Acknowledgments

This training program was developed with contributions from the following teams and individuals:

- Engineering Excellence Team - Curriculum development and technical review
- Global Learning & Development - Instructional design and assessment framework
- AI/ML Center of Excellence - AI integration modules and best practices
- Regional Training Teams - Localization and delivery adaptation
- Previous Program Cohorts - Feedback and continuous improvement

END OF DOCUMENT

XSPARKS

Engineering Excellence Division